

Session 4

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 17-Sep-2026

Announcements

- Reminder — if you have not yet, finish the Module 1 capstone notebook (all four skills together):
[m1_practice_exercises_student.ipynb](#)

Previously on CRB1

- Session 3: Refresher Module 1 (PDF) — the tabular model (Pandas), the PyTorch bridge
- Notebooks 3–4 (Pandas, Matplotlib)

Questions?

Today's Agenda

- Refresher Module 2 (PDF): vector spaces & bases, change of basis in the Γ notation, the conjugation trick, norms, projection, Gram–Schmidt, the transformation zoo, eigen/SVD, least squares, PCA
- Notebook 1 — Vectors & Transformations
 - independence and basis, the scalar product, norms, projection, Gram–Schmidt, the zoo, change of basis, polynomials and matrices as vector spaces
- Notebook 2 — Eigen, SVD & PCA
 - eigenvectors, diagonalisation, the SVD, least squares as a projection, the pseudoinverse, ridge, PCA
- Kahoot

Refresher Modules

Where today's session fits before Part I begins

- Module 1: Python, NumPy, Pandas, Matplotlib — the data & compute layer (Sessions 2–3)
 - Module 2: Linear Algebra — vectors, transforms, eigen/SVD, PCA (today)
 - Module 3: Calculus — derivative, gradient, gradient descent, autodiff (Session 5); integration, PID (Session 22)
 - Module 4: Image Processing (OpenCV) (Session 6)
 - Module 5: LLMs & AI Coding Assistants (Session 7)
-
- Every model is a chain of vector/matrix operations — the geometry Géron's Chapters 4 & 7 run on, and it reappears as SO(3) in Robotics and the eigen-machinery in Control

3Blue1Brown

https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab

- Vectors. Definition.
- Linear combinations, basis, span
- “Row” and “Column” vectors
- Basis change
- Orthonormalization
- Concept of “linear”. Linear transformations. Matrices. Basis change in linear transformations.
- Projections, Reflections, Rotations
- Composition, inversion, solving linear equations
- Eigenvectors and eigenvalues

Vector Spaces & Basis

Refresher Module 2 (PDF), §1–3 — what a vector space actually is

- A vector space is any set with addition & scalar multiplication obeying ten familiar rules (closure, associativity, an identity, an inverse, distributivity) — $\mathbb{R}^n$ is the example we compute in, but matrices and polynomials qualify too
- A linear combination is a weighted sum of vectors; the span of a set is everything reachable that way, by scaling and adding
- A basis is a linearly independent spanning set — the minimal information needed to reach every vector; it is never unique, and its size is the space's dimension
- In practice you test independence with the rank: wire three electrodes so one lead is the difference of the other two and you record a rank-2 cloud, however long you record for

Change of Basis (Γ)

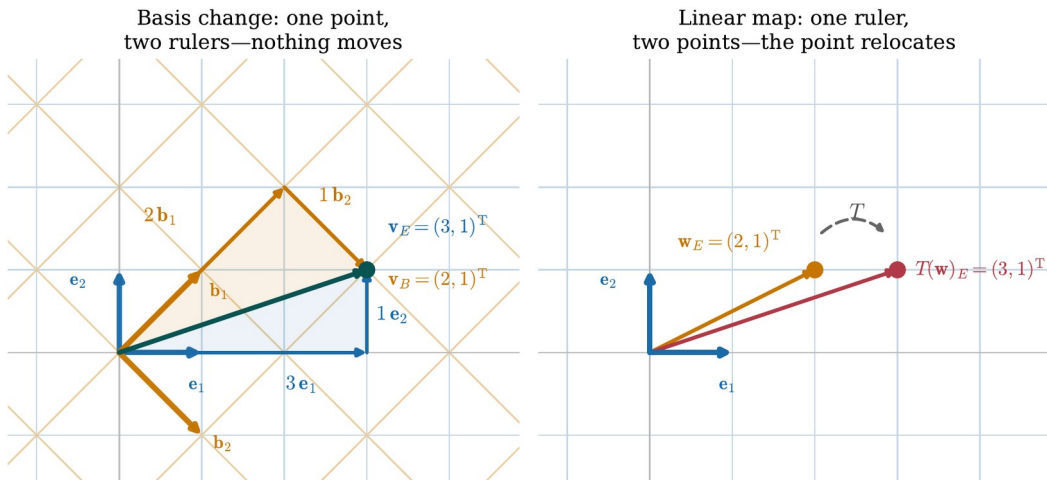
Refresher Module 2 (PDF), §4 — same point, different coordinates

- Row/column vectors are just coordinates in the standard basis $e_1, \dots, e_n$ — coordinates are never absolute, always relative to a chosen basis
- Write each vector of E in B-coordinates and stack the columns: that is Γ_{BE} , and $v_B = \Gamma_{BE} v_E$. Read the subscript left to right — Γ_{BE} expresses the second letter's vectors in the first letter's basis
- $\Gamma_{EB} = (\Gamma_{BE})^{-1}$, and when E is canonical Γ_{EB} is simply the new basis vectors as columns
- A map changes the same way: $A_E = \Gamma_{EB} A_B \Gamma_{BE}$, a similarity transform — exactly what PCA does when it rotates onto the principal components

Same Matrix, Two Meanings

Refresher Module 2 (PDF), §8.1 — a basis change is not a linear map

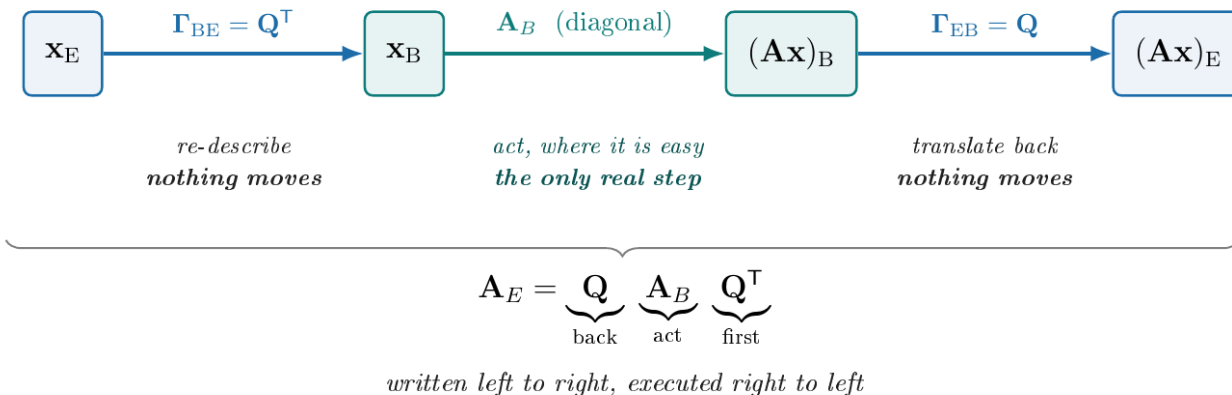
- The recipes coincide: Γ_{EB} collects the new basis vectors in E-coordinates as columns; a map's matrix collects the images $T(e_i)$ in E-coordinates as columns
- A basis change leaves the vector alone and changes its description — one arrow, two coordinate lists. A linear map leaves the basis alone and moves the vector
- Same matrix, same input numbers, same output numbers — and two different true statements. Only one difference survives: a basis-change matrix is always invertible, a map's need not be



Choosing an Easy Basis

Refresher Module 2 (PDF), §8.2 — the trick behind most of this module (Notebook 1, §13.2)

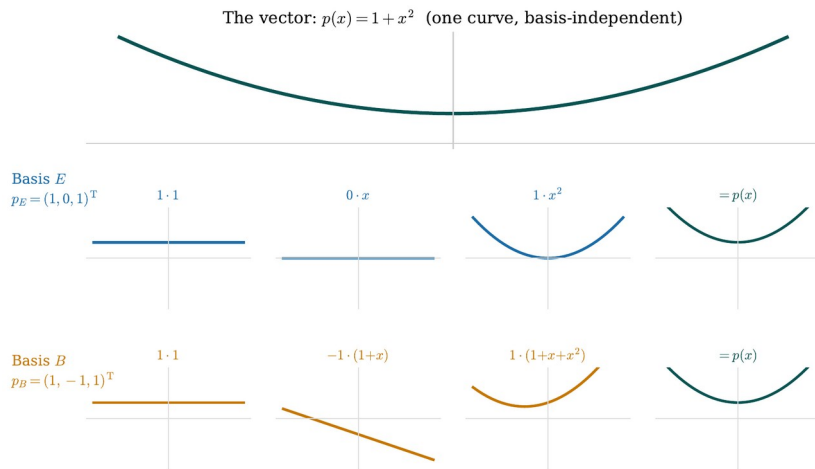
- Changing basis changes a map's matrix, $A_E = \Gamma_{EB} A_B \Gamma_{BE}$. That reads as bad news until you notice the corollary: you get to choose the basis
- So choose one adapted to the map — basis vectors along the mirror line, the rotation axis, the subspace you project onto. In that basis A_B is diagonal and you write it down by inspection. Then transport it back with $A_E = Q A_B Q^T$ — and if B is orthonormal there is no inverse to compute, only a transpose
- You never compute the canonical matrix; you assemble it. Do this for a rotation about an arbitrary axis and out falls Rodrigues' formula — 120° about $(1,1,1)$ gives the cyclic permutation matrix exactly. Eigendecomposition, SVD, PCA and the least-squares projector are all the same move, differing only in how B is found



Vector Spaces Beyond $\mathbb{R}^n$

Refresher Module 2 (PDF), §5 — polynomials and 2×2 matrices

- Nothing so far used the fact that a vector is an arrow. $P_2 = \{a_0 + a_1x + a_2x^2\}$ is a 3-dimensional vector space; $\mathbb{R}^{2 \times 2}$ is a 4-dimensional one
- Both get two bases and the very same Γ_{BE} machinery — the arithmetic is identical, which is the whole point
- For a 2-D displacement-gradient tensor the second basis is physically meaningful: the symmetric part is the strain and the antisymmetric part is the rigid rotation, read off as separate coordinates

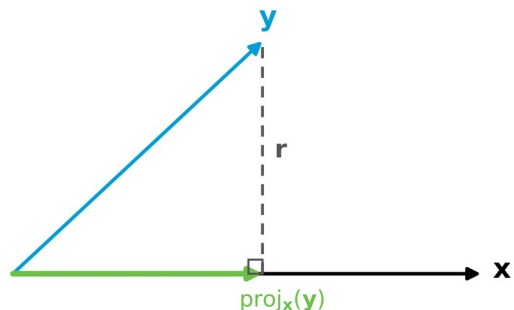


Vectors & Projection

Notebook 1, §2–6 — the scalar product and the geometry of projection

- The scalar product needs the Gram matrix $G_{ij} = \langle e_i, e_j \rangle$: $\langle v, w \rangle = v^T G w$ in general, and collapses to the familiar dot product exactly when the basis is orthonormal ($G = I$)
- In $\mathbb{R}^2/\mathbb{R}^3$, $x \cdot y = |x||y|\cos(\theta)$ is a theorem (Law of Cosines) — θ is the ordinary, protractor-measured angle
- In higher dimensions there is no protractor: $\cos(\theta) := \langle x, y \rangle / (|x||y|)$ becomes the definition instead — valid because Cauchy–Schwarz keeps the ratio in $[-1, 1]$

Orthogonal Projection



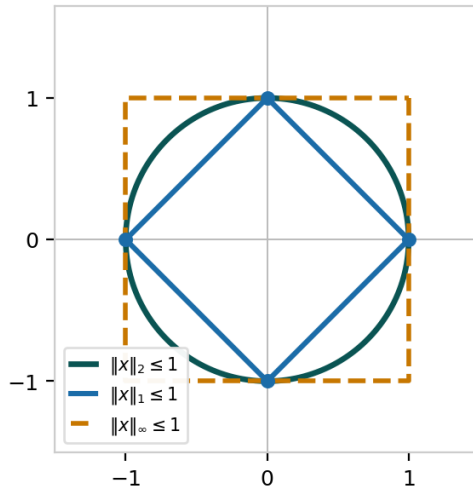
Any vector splits into a part along x and a part perpendicular to it

Norms: What Is a Length?

Refresher Module 2 (PDF), §6.3 — why only L2 gives you angles

- A norm is any rule obeying three axioms: positivity, homogeneity $\|cv\| = |c|\|v\|$, and the triangle inequality $\|v+w\| \leq \|v\| + \|w\|$. L1, L2 and L^∞ all qualify
- Only L2 comes from an inner product — the test is the parallelogram law. So angles, orthogonality, projection, least squares and Gram–Schmidt are privileges of the scalar product, not of merely having a length

only the round one comes from a scalar product

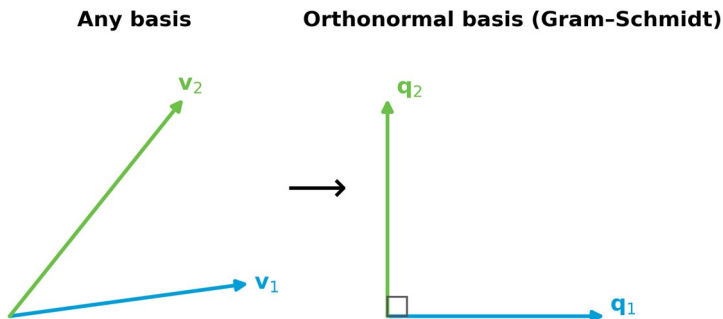


Gram–Schmidt & the Zoo

Notebook 1, §7–12 — an orthonormal basis, then matrices read as verbs

- Gram–Schmidt turns any independent set into an orthonormal one using nothing but the projection of the previous slide, repeated against a growing subspace — and stacking the result gives the QR factorisation real solvers use
- The columns of A are where it sends e_1 and e_2 , so a matrix is a verb: scaling, rotation, reflection, shear, projection. Composition is multiplication, right-to-left, and it does not commute

Gram–Schmidt Orthonormalization



Subtract the projection onto every axis built so far, then normalise

Let's practice — Notebook 1

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m2_vectors_transforms_student.ipynb

[Open Notebook 1 in Google Colab](#) →

Eigenvectors & the SVD

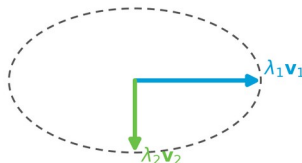
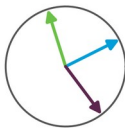
Notebook 2, §1–5 — the invariant axes, and the factorisation behind everything

- An eigenvector is a direction the map only stretches, $Av = \lambda v$. Stack them and $A = P\Lambda P^{-1}$ — in the eigenbasis the map is pure scaling. For symmetric matrices the eigenvectors are orthonormal (the spectral theorem), which is the case ML uses most
- The SVD generalises it to any $m \times n$ matrix: $A = U\Sigma V^T$ is rotate $\rightarrow$ scale $\rightarrow$ rotate. Keeping the top k singular values gives the best rank- k approximation (Eckart–Young) — compression, denoising, and PCA all at once

Eigenvectors Are the Map's Invariant Axes

Most directions rotate

Eigenvectors: $Av = \lambda v$ only stretches



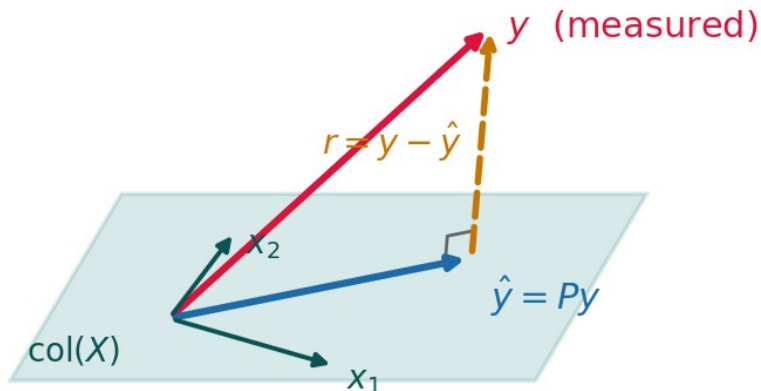
The circle maps to an ellipse whose axes are the eigenvectors, scaled by their eigenvalues

Least Squares = Projection

Refresher Module 2 (PDF), §14.1 and Annex A — the geometry behind the normal equation

- With more equations than unknowns there is no exact fit, so choose w making the residual orthogonal to every column of X : $X^T X w = X^T y$, the normal equation (Géron Ch. 4)
- Substituting back makes the projection explicit: $\hat{y} = Py$ with the hat matrix $P = X(X^T X)^{-1} X^T$ — symmetric, idempotent, and of rank equal to the number of features
- With one column it collapses to the projection formula character for character. One column is a line, n columns are a subspace, and the operator is the same

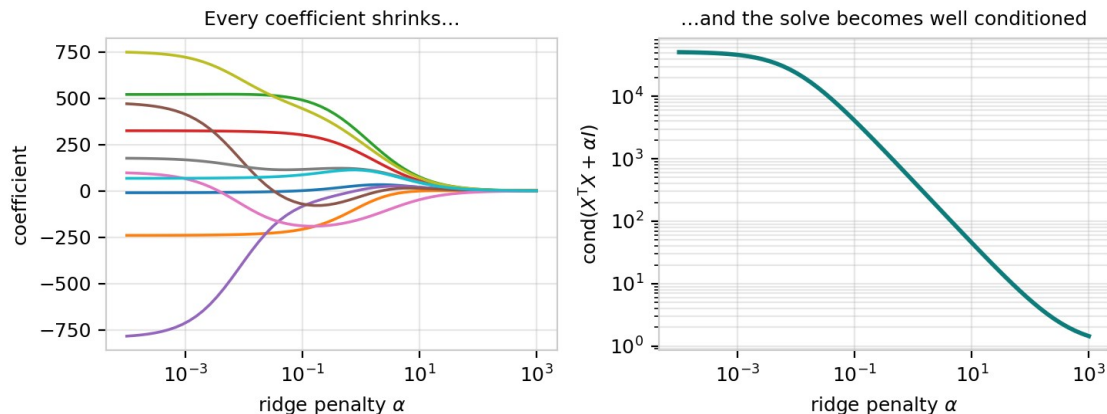
the residual stands perpendicular to every column of X



Pseudoinverse & Ridge

Refresher Module 2 (PDF), §14.1 — what ltsq actually calls, and what α does

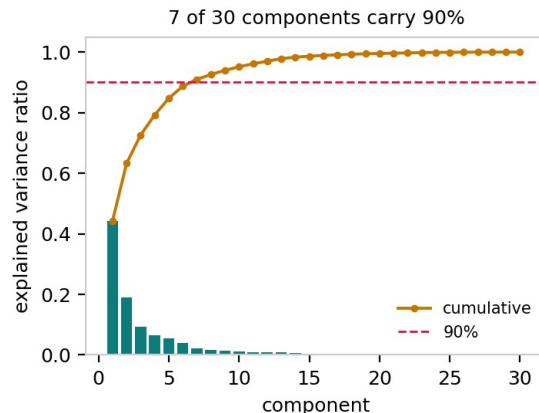
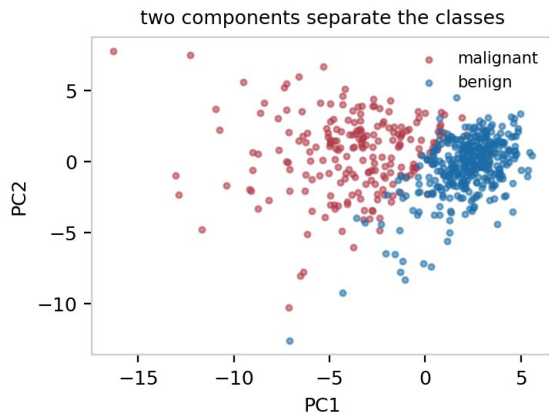
- Never form $(X^T X)^{-1}$: squaring X squares its condition number. The SVD route gives $X^+ = V \Sigma^+ U^T$, and $w = X^+ y$ — exactly what `np.linalg.lstsq` and Scikit-Learn's `LinearRegression` compute
- X^+ survives rank deficiency: when infinitely many w fit equally well it returns the one of smallest norm instead of failing
- Ridge adds $\alpha \|w\|_2^2$: $w = (X^T X + \alpha I)^{-1} X^T y$. Read through the spectral theorem it shifts every eigenvalue to $\lambda_i + \alpha$ — invertible whatever the data did, better conditioned, and shrinking hardest exactly the directions the data does not constrain



PCA, Done Properly

Refresher Module 2 (PDF), §14.2 — three things that have to be right

- Centre first — this is not optional. PCA measures variance about the mean; skip it and the first component points at where the cloud sits rather than how it is shaped. Standardise too, or the feature with the largest units wins by default
- The object being diagonalised is the covariance $C = (1/m)X^TX$ — symmetric and positive semi-definite, so the spectral theorem hands you an orthonormal eigenbasis
- The SVD gets there without forming C : the components are the columns of V and $\lambda_i = \sigma_i^2/m$. Projection is XV_k , reconstruction is $XV_kV_k^T$, and the discarded variance is the error



Let's practice — Notebook 2

https://github.com/SergiSama/UIC-CRB1-2026-2027/blob/main/m2_eigen_svd_pca_student.ipynb

[Open Notebook 2 in Google Colab](#) →

Before Next Session

- Finish both notebooks' exercises (solution cells included — check your own work)
- Re-read Refresher Module 2 (PDF) if you haven't already — §5, §6.3, §8.1–8.2 and §14 are new
- Session 5 preview: PageRank, then Calculus Part 1 (Module 3 PDF)

Q & A



End of Session 4

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 17-Sep-2026